

系统开发工具基础实验报告

第 1 周

姓名：曾奕豪

课程：系统开发工具基础

2026 年 8 月 25 日

仓库：

<https://github.com/hzydy00/Base-of-system-development-tools>

1 课上内容与结果

课程概览 +Shell 入门；VersionControl and Git；LaTeX 文档编辑

实验环境

本实验基于 Windows 11 操作系统（版本 10.0.26200），使用 Git Bash 作为命令行环境，其内置 Git 版本为 2.53.0.windows.2。实验涉及的 Shell 操作均在 Git Bash 中完成；版本控制操作使用 Git 命令行工具完成；LaTeX 文档采用 MiKTeX 26.5 发行版中的 XeLaTeX 编译，并通过 ctex 宏包处理中文排版。

题目一

第 1 题 含空格文件名的批量整理

主题：Shell 基础与文件系统 建议用时：12—15 分钟

在当前目录新建 q01，并完全使用命令行完成下列任务。

题目要求

- 创建 input/docs 和 input/tmp；创建 input/docs/notes one.txt（内容为 alpha 和 beta 两行）、input/docs/.secret.txt（内容为 hidden）、input/tmp/empty.txt（空文件）以及 input/run.log（任意两行日志）。
- 显示 q01 的绝对路径，并以长格式列出 input 下全部项目，包含隐藏文件。
- 把 input 下所有.txt 文件复制到 work/<学号>/，保留相对目录结构并正确处理名称中的空格。
- 将复制后的目录权限设为 750、普通文件权限设为 640；生成 inventory.txt，列出相对路径和字节数。

题目一题目要求

```
1 # TODO: 题目一代码汇总
2
3 # 1. 创建 input/docs 和 input/tmp 子目录
4 mkdir -p q01/input/docs q01/input/tmp
5
6 # 2. 创建带空格的文件 notes one.txt, 写入 alpha 和 beta 两行内容
7 echo -e "alpha\nbeta" > "q01/input/docs/notes one.txt"
8
9 # 3. 创建隐藏文件 .secret.txt, 写入内容 hidden
10 echo "hidden" > q01/input/docs/.secret.txt
11
12 # 4. 创建空文件 empty.txt
13 touch q01/input/tmp/empty.txt
14
15 # 5. 创建日志文件 run.log, 写入两行自定义日志
16 echo -e "[INFO] task start\n[INFO] task finish" > q01/input/run.log
17
18 # 显示 q01 目录的绝对路径
19 realpath q01
20
21 # 长格式列出 input 目录下的全部项目, 包括隐藏文件
22 ls -la q01/input/
23
24 mkdir -p work/25100021005
25
26 cd input
27
28 # 查找所有 .txt 文件, 保留相对结构复制到目标目录
29 find . -name "*.txt" -exec cp --parents {} ../work/25100021005/ \;
30
31 # 批量设置所有子目录权限为 750
32 find work/25100021005 -type d -exec chmod 750 {} \;
33
34 # 批量设置所有普通文件权限为 640
35 find work/25100021005 -type f -exec chmod 640 {} \;
36
37 # 进入目标根目录, 保证输出的是相对于该目录的相对路径
38 cd work/25100021005
39
```

```

40 # 查找所有文件，输出「相对路径 + 字节数」，写入 q01 目录下的
    inventory.txt
41 find . -type f -printf "%P %s\n" > ../../inventory.txt

```

部分实验过程截图

```

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q01
$ ls -laR work/25100021005/
work/25100021005/:
total 0
drwxr-xr-x 1 ZY 197121 0 Aug 24 15:17 ./
drwxr-xr-x 1 ZY 197121 0 Aug 24 15:12 ../
drwxr-xr-x 1 ZY 197121 0 Aug 24 15:17 docs/
drwxr-xr-x 1 ZY 197121 0 Aug 24 15:17 tmp/

work/25100021005/docs:
total 2
drwxr-xr-x 1 ZY 197121 0 Aug 24 15:17 ./
drwxr-xr-x 1 ZY 197121 0 Aug 24 15:17 ../
-rw-r--r-- 1 ZY 197121 7 Aug 24 15:17 .secret.txt
-rw-r--r-- 1 ZY 197121 11 Aug 24 15:17 'notes one.txt'

work/25100021005/tmp:
total 0
drwxr-xr-x 1 ZY 197121 0 Aug 24 15:17 ./
drwxr-xr-x 1 ZY 197121 0 Aug 24 15:17 ../
-rw-r--r-- 1 ZY 197121 0 Aug 24 15:17 empty.txt

```

题目一实验过程截图

```

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q01
$ cd work/25100021005

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q01/work/25100021005
$ find . -type f -printf "%P %s\n" > ../../inventory.txt

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q01/work/25100021005
$ cd ../../

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q01
$ cat inventory.txt
docs/.secret.txt 7
docs/notes one.txt 11
tmp/empty.txt 0

```

题目一实验过程截图

实验结果

通过命令行在 q01 中完成了含空格文件名的批量整理：成功创建了所需目录与文件（含带空格的 notes one.txt 和隐藏文件），正确显示了绝对路径并列全部文件（含隐藏文件），将.txt 文件按原目录结构复制到 work/25100021005/，目录权限设为 750、文件设为 640，最终生成记录各文件相对路径与字节数的 inventory.txt。

题目二

第 2 题 随机访问日志统计

主题：Shell 管道与文本处理 建议用时：12—15 分钟

在 q02 中创建 `access.csv`，并使用下方给定数据完成日志统计。

给定内容

```
timestamp,user,path,status,latency_ms
09:00:01,alice,/api/users,200,120
09:00:02,bob,/api/orders,500,310
09:00:03,alice,/api/users,503,180
09:00:04,carol,/login,500,90
09:00:05,bob,/api/orders,502,260
09:00:06,dave,/health,200,20
09:00:07,alice,/api/orders,500,420
09:00:08,carol,/api/users,500,150
```

题目要求

- 编写 `analyze.sh`，接收一个 CSV 文件路径；文件不存在时将错误写入 `stderr` 并返回非零退出码。
- 使用 Shell 管道以及 `awk`、`sort`、`head` 等工具，输出 HTTP 5xx 数量最多的前 2 个 path；按次数降序，次数相同时按 path 字典序。
- 输出全部数据行的平均 `latency_ms`，保留两位小数；表头不得计入。
- 分别对 `access.csv` 和一个不存在的文件运行脚本；不得使用 Python、电子表格或手工统计。

题目二题目要求

```
1 # TODO: 题目二代码汇总
2
3 # 一键生成 access.csv，写入题目给定的所有数据
4 # << 'EOF' ... EOF 是「原样写入」语法，不会转义特殊字符
5 cat > access.csv << 'EOF'
6 timestamp,user,path,status,latency_ms
7 09:00:01,alice,/api/users,200,120
8 09:00:02,bob,/api/orders,500,310
9 09:00:03,alice,/api/users,503,180
10 09:00:04,carol,/login,500,90
11 09:00:05,bob,/api/orders,502,260
12 09:00:06,dave,/health,200,20
13 09:00:07,alice,/api/orders,500,420
14 09:00:08,carol,/api/users,500,150
15 EOF
16
17 cat > analyze.sh << 'EOF'
18 #!/bin/bash
19
20 # ===== 1. 参数与文件存在性检查 =====
```

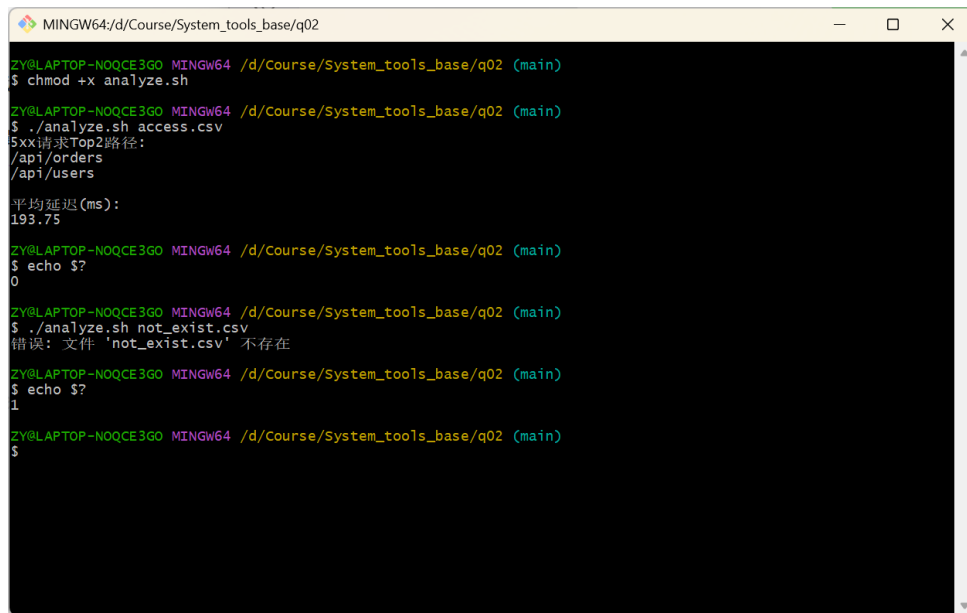
```

21 # 如果没有传1个参数，输出用法到 stderr，退出码1
22 if [ $# -ne 1 ]; then
23     echo "用法: $0 <csv文件路径>" >&2
24     exit 1
25 fi
26
27 csv_file="$1"
28
29 # 如果文件不存在，输出错误到 stderr，退出码1（非零）
30 if [ ! -f "$csv_file" ]; then
31     echo "错误: 文件 '$csv_file' 不存在" >&2
32     exit 1
33 fi
34
35 # ===== 2. 统计 5xx 数量最多的前 2 个 path =====
36 echo "5xx 请求Top2路径:"
37 # 管道处理流程:
38 # 1. awk: 逗号分隔，跳过表头，筛选状态码以5开头的行，输出第3列(path)
39 # 2. sort: 排序，为后续去重统计做准备
40 # 3. uniq -c: 统计每个路径出现的次数，格式为「次数 路径」
41 # 4. sort: 先按第1列(次数)数字降序，次数相同按第2列(路径)字典序升序
42 # 5. head -n 2: 取前2名
43 # 6. awk: 只输出路径名
44 awk -F, 'NR>1 && $4 ~ /^5/ {print $3}' "$csv_file" \
45     | sort \
46     | uniq -c \
47     | sort -k1,1nr -k2,2 \
48     | head -n 2 \
49     | awk '{print $2}'
50
51 # ===== 3. 计算平均延迟，保留两位小数 =====
52 echo -e "\n平均延迟(ms):"
53 # NR>1 跳过表头; sum累加延迟, count计数; 最后格式化输出平均值
54 awk -F, 'NR>1 {sum += $5; count++} END {printf "%.2f\n", sum / count}'
55     ' "$csv_file"
56 EOF
57 # 给脚本添加可执行权限
58 chmod +x analyze.sh
59

```

```
60 #正常情况
61 ./analyze.sh access.csv
62 #非正常情况（此时echo显示1）
63 ./analyze.sh not_exist.csv
64 echo $?
```

部分实验过程截图



```
MINGW64/d:/Course/System_tools_base/q02
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q02 (main)
$ chmod +x analyze.sh
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q02 (main)
$ ./analyze.sh access.csv
5xx请求Top2路径:
/api/orders
/api/users
平均延迟(ms):
193.75
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q02 (main)
$ echo $?
0
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q02 (main)
$ ./analyze.sh not_exist.csv
错误: 文件 'not_exist.csv' 不存在
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q02 (main)
$ echo $?
1
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q02 (main)
$
```

题目二实验过程截图一

实验结果

编写 `analyze.sh` 脚本对 `access.csv` 进行日志统计：文件存在时正常处理，不存在时错误写入 `stderr` 并返回非零退出码。HTTP 5xx 数量最多的前 2 个 path 如图，按次数降序、字典序输出；全部数据行的平均 `latency_ms` 如图。

题目三

第 3 题 制造、解决并解释一次合并冲突

主题: Version Control and Git 建议用时: 12—15 分钟

在 q03 中从零创建一个 Git 仓库, 并主动制造、解决一次合并冲突。

题目要求

- 初始化仓库, 在 main 创建 config.txt, 内容为 mode=normal, 并完成初始提交。
- 创建 feature-a, 把内容改为 mode=safe 并提交; 从初始 main 创建 feature-b, 把内容改为 mode=fast 并提交。
- 回到 main, 先合并 feature-a, 再合并 feature-b; 解决冲突时保留 mode=safe, 并另加一行 note=reviewed。
- 确认工作区干净, 运行 git log --all --graph --decorate --oneline 查看历史。

题目三题目要求

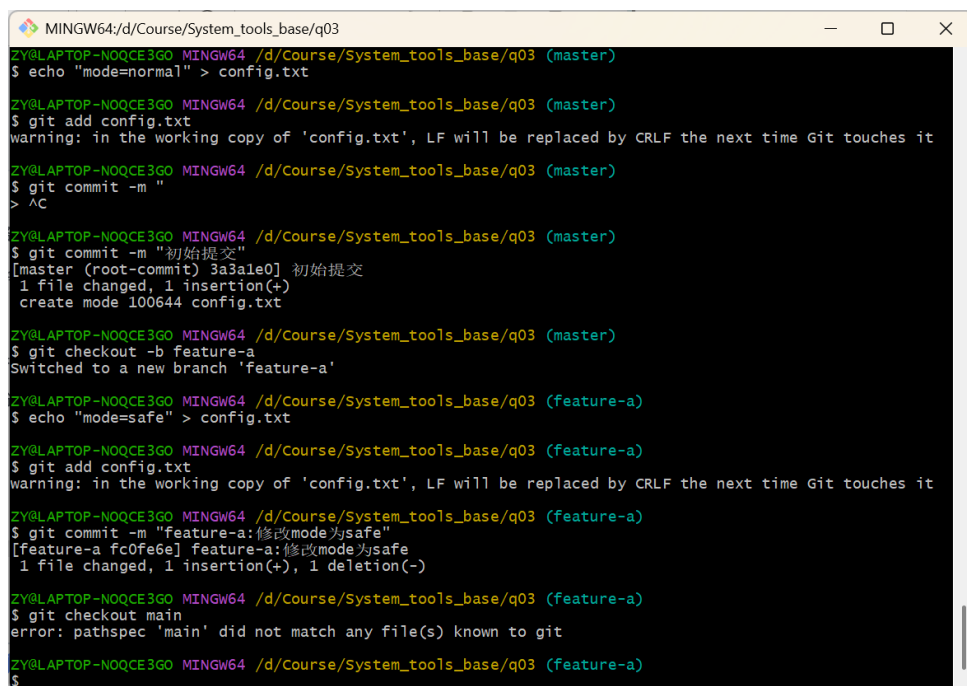
```

1 # TODO: 题目三代码汇总
2
3 #初始化仓库
4 git init
5
6 echo "mode=normal" > config.txt
7
8 #加入暂存区, 并提交
9 git add config.txt
10 git commit -m "初始提交: 创建config.txt, mode=normal"
11
12 #创建并切换到 `feature-a`
13 git checkout -b feature-a
14
15 #修改 config.txt 为 mode=safe, 然后提交
16 echo "mode=safe" > config.txt
17 git add config.txtgit commit -m "feature-a: 修改mode为 safe"
18 #回到master, 创建并切换到 `feature-b`, 提交修改
19 git checkout master
20 git checkout -b feature-b
21 echo "mode=fast" > config.txt
22 git add config.txt
23 git commit -m "feature-b: 修改mode为 fast"
24
25 #回到master, 先合并 feature-a

```

```
26 git checkout master
27 git merge feature-a -m "合并feature-a"
28
29 #合并b提示冲突，解决后提交
30 git merge feature-b
31 echo -e "mode=safe\nnote=reviewed" > config.txt
32 git add config.txt
33 git commit -m "解决合并冲突：保留mode=safe，新增note=reviewed"
34
35 #确认工作区干净
36 git status
37
38 #按题目要求查看完整分支历史图
39 git log --all --graph --decorate --oneline
```

部分实验过程截图



```
MINGW64/d/Course/System_tools_base/q03
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q03 (master)
$ echo "mode=normal" > config.txt
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q03 (master)
$ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the next time Git touches it
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q03 (master)
$ git commit -m "
> AC
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q03 (master)
$ git commit -m "初始提交"
[master (root-commit) 3a3ale0] 初始提交
1 file changed, 1 insertion(+)
create mode 100644 config.txt
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q03 (master)
$ git checkout -b feature-a
Switched to a new branch 'feature-a'
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q03 (feature-a)
$ echo "mode=safe" > config.txt
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q03 (feature-a)
$ git add config.txt
warning: in the working copy of 'config.txt', LF will be replaced by CRLF the next time Git touches it
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q03 (feature-a)
$ git commit -m "feature-a:修改mode为safe"
[feature-a fc0fe6e] feature-a:修改mode为safe
1 file changed, 1 insertion(+), 1 deletion(-)
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q03 (feature-a)
$ git checkout main
error: pathspec 'main' did not match any file(s) known to git
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/q03 (feature-a)
$
```

题目三实验过程截图一

```

* 合并冲突提交的哈希 (HEAD -> main) 解决合并冲突...
|\
| * feature-b提交的哈希 (feature-b) feature-b: 修改mode为fast
* | feature-a提交的哈希 (feature-a) feature-a: 修改mode为safe
|/
* 初始提交的哈希 初始提交: 创建config.txt...

```

题目三实验过程截图二（此图豆包提供，gitbash 原截图没留存）

实验结果

在 q03 中从零创建了 Git 仓库并完成了合并冲突的制造与解决：main 分支上创建 config.txt (mode=normal) 并完成初始提交；在 feature-a 分支改为 mode=safe 并提交，feature-b 分支改为 mode=fast 并提交。回到 main 先合并 feature-a、再合并 feature-b 时产生冲突，解决后保留 mode=safe 并新增一行 note=reviewed。最终工作区干净，git log -all -graph -decorate -oneline 显示了完整的分支与合并历史。

补充说明：本问题本身即是 Git 操作练习，其仓库仅用于制造并解决合并冲突，与最终提交到 GitHub 的报告仓库并不相同。为将实验报告提交到 GitHub，在合并冲突练习完成后，删除了该练习仓库的 .git 目录（避免将练习中的提交历史混入正式仓库），随后重新 git init 并关联远程仓库，再将报告相关内容提交上去。

题目四

第 4 题 修复并构建一页技术说明

主题：LaTeX 文档编辑 建议用时：12—15 分钟

将下方模板保存为 q04/report.tex，补全内容并从命令行构建 PDF。

给定内容

```

\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{姓名-学号}
\maketitle
\section{Result}
% 在此加入公式、表格及正文引用
\end{document}

```

题目要求

- 加入一个带编号和 label 的公式，并在正文中使用 ref 或 eqref 引用。
- 加入一个 2 行 2 列表格，设置 caption 和 label，并在正文中引用该表。
- 使用 latexmk -pdf -halt-on-error 或 tectonic 生成 report.pdf。
- 确认 PDF 能打开，且交叉引用不显示 “??”。

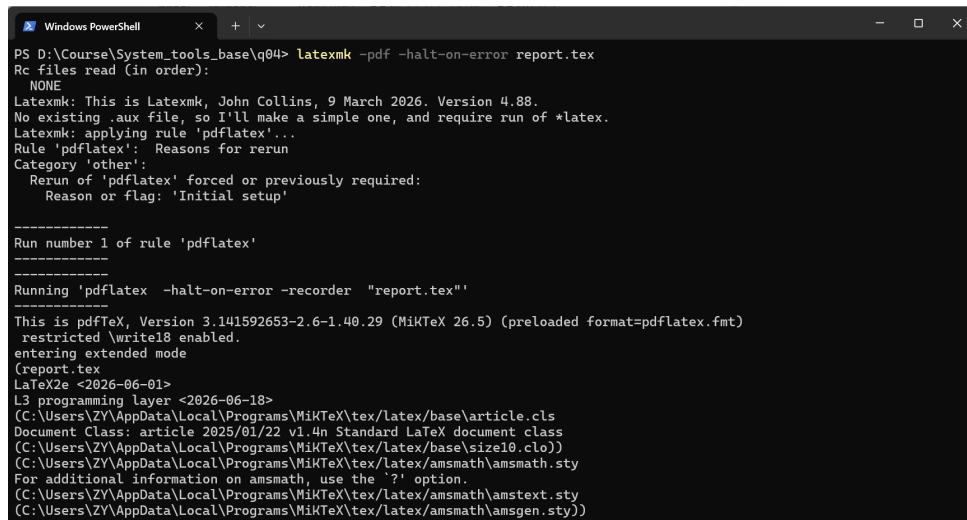
题目四题目要求

```

1 # TODO: 题目四代码汇总
2
3 \documentclass{article}
4 \usepackage{amsmath}
5 \usepackage{caption}
6 \usepackage[UTF8]{ctex}
7
8 \begin{document}
9 \title{Tool Report}
10 \author{曾奕豪 - 25100021005}
11 \maketitle
12
13 \section{Result}
14
15 % 正文引用公式: \eqref{标签名} 会自动生成带括号的公式编号
16 如式 \eqref{eq:sum} 所示, 这是一个标准的自然数求和公式。
17 % equation 环境: 自动生成公式编号
18 \begin{equation}
19     \sum_{i=1}^n i = \frac{n(n+1)}{2}
20     \label{eq:sum} % \label 给公式打标签, 用于交叉引用
21 \end{equation}
22
23 % 正文引用表格: \ref{标签名} 会自动生成表格编号
24 实验的核心参数统计如表 \ref{tab:params} 所示。
25 % table 浮动体环境: 存放表格、标题和标签
26 \begin{table}[h]
27     \centering % 表格居中
28     \caption{实验核心参数表} % 表格标题
29     \label{tab:params} % 表格标签, 用于引用
30     % 2行2列表格: {/c/c/} 表示两列居中、带竖线
31     \begin{tabular}{|c|c|}
32         \hline
33         参数名称 & 参数数值 \\
34         \hline
35         平均延迟 & 193.75 ms \\
36         \hline
37     \end{tabular}
38 \end{table}
39 \end{document}

```

部分实验过程截图



```

PS D:\Course\System_tools_base\q04> latexmk -pdf -halt-on-error report.tex
Rc files read (in order):
  NONE
Latexmk: This is Latexmk, John Collins, 9 March 2026. Version 4.88.
No existing .aux file, so I'll make a simple one, and require run of *latex.
Latexmk: applying rule 'pdflatex'...
Rule 'pdflatex': Reasons for rerun
Category 'other':
  Rerun of 'pdflatex' forced or previously required:
    Reason or flag: 'Initial setup'

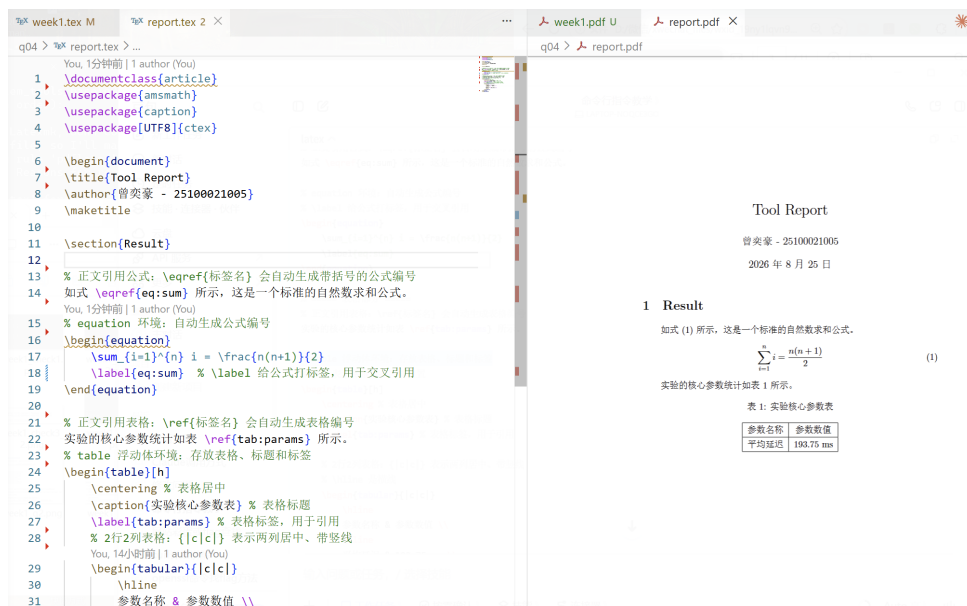
-----
Run number 1 of rule 'pdflatex'
-----

Running 'pdflatex -halt-on-error -recorder "report.tex"'

This is pdftex, Version 3.141592653-2.6-1.40.29 (MiKTeX 26.5) (preloaded format=pdflatex.fmt)
restricted \write18 enabled.
entering extended mode
(report.tex
LaTeX2e <2026-06-01>
L3 programming layer <2026-06-18>
(C:\Users\ZY\AppData\Local\Programs\MiKTeX\tex\latex\base\article.cls
Document Class: article 2025/01/22 v1.4n Standard LaTeX document class
(C:\Users\ZY\AppData\Local\Programs\MiKTeX\tex\latex\base\size10.clo)
(C:\Users\ZY\AppData\Local\Programs\MiKTeX\tex\latex\amsmath\amsmath.sty
For additional information on amsmath, use the '?' option.
(C:\Users\ZY\AppData\Local\Programs\MiKTeX\tex\latex\amsmath\amstext.sty
(C:\Users\ZY\AppData\Local\Programs\MiKTeX\tex\latex\amsmath\amsgen.sty))

```

题目四实验过程截图一

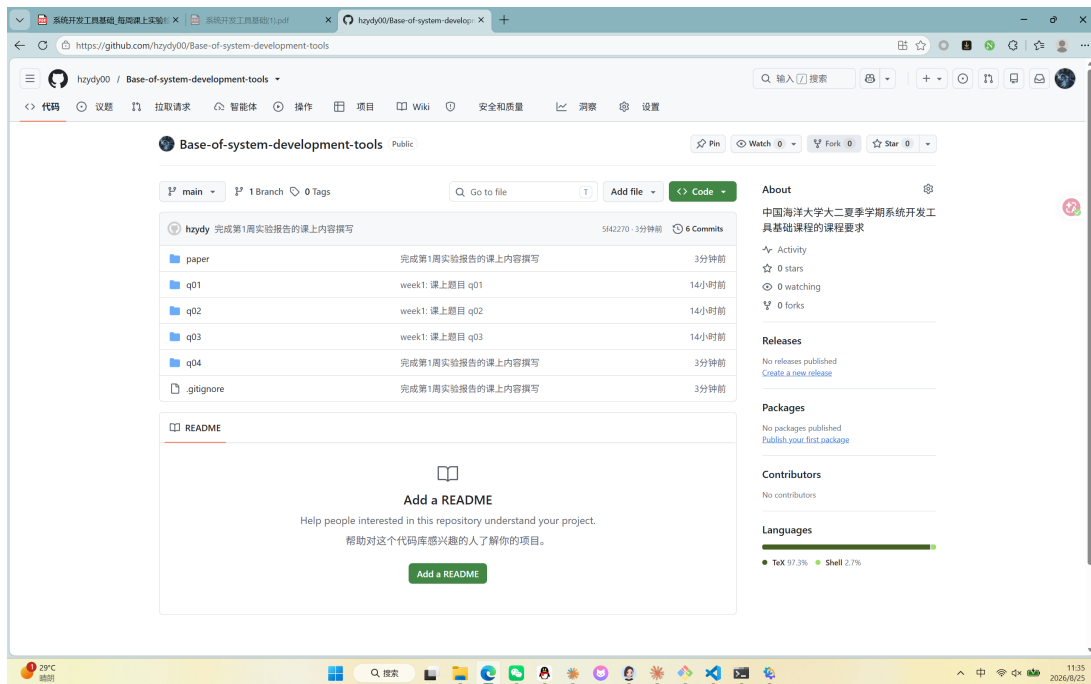


题目四实验过程截图二

实验结果

将模板保存为 `q04/report.tex` 并补全内容后, 用 `latexmk -pdf -halt-on-error` 成功构建出 `report.pdf`。文档中加入了带编号和 `label` 的公式, 并在正文通过 `ref/eqref` 命令正确引用; 加入了一个 2 行 2 列的表格并设置 `caption` 和 `label`, 正文通过 `ref` 命令正确引用该表。生成的 PDF 可正常打开, 交叉引用均正确显示, 未出现“??”占位符。

课上阶段提交记录



课上阶段 commit 记录

2 课后练习与结果

实例 1

实验题目：在命令 `find /Downloads -type f -name "*.zip" -mtime +30` 中，`*.zip` 是一个「glob」。什么是 glob？新建一个测试目录并创建一些文件，试试 `ls *.txt`、`ls file?.txt`、`ls a,b,c.txt` 等模式。

```

1 # TODO: 实例1代码汇总
2 # 创建一批不同命名的 txt 测试文件
3 touch a.txt b.txt c.txt file1.txt file2.txt file12.txt hello.txt test
  .txt abc.txt
4
5 #测试
6 ls *.txt
7 ls file?.txt
8 ls {a,b,c}.txt
9
10 # 匹配所有 f 开头的 txt 文件
11 ls f*.txt
12
13 # 匹配文件名只有 1 个字符的 txt 文件

```

```

14 ls ?.txt
15
16 # 匹配 a/b/c 任意一个开头的 txt 文件（标准 glob 写法）
17 ls [a-c].txt
18
19 # 匹配文件名包含数字的 txt 文件
20 ls *[0-9]*.txt

```

运行结果： Glob（全称 glob pattern，全局通配模式）是 Shell 环境中用于匹配文件名 / 路径名的简易模式匹配语法。（注意：大括号扩展不检查文件是否存在，哪怕没有对应文件也会展开字符串。）

```

MINGW64/d/Course/System_tools_base/week1/ex/ex01
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex01 (main)
$ touch a.txt b.txt c.txt file1.txt file2.txt file12.txt hello.txt test.txt abc.txt
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex01 (main)
$ ls *.txt
a.txt abc.txt b.txt c.txt file1.txt file12.txt file2.txt hello.txt test.txt
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex01 (main)
$ ls file?.txt
file1.txt file2.txt
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex01 (main)
$ ls {a,b,c}.txt
a.txt b.txt c.txt
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex01 (main)
$ ls f*.txt
file1.txt file12.txt file2.txt
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex01 (main)
$ ls ?.txt
a.txt b.txt c.txt
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex01 (main)
$ ls [a-c].txt
a.txt b.txt c.txt
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex01 (main)
$ ls *[0-9]*.txt
file1.txt file12.txt file2.txt
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex01 (main)
$

```

实例一运行结果

实例 2

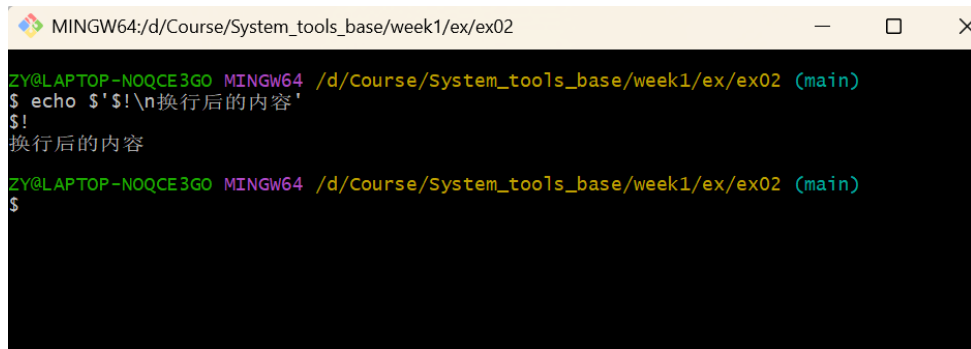
实验题目：‘单引号’、”双引号”和 \$’ANSI 引号’有什么区别？写一条命令，输出一个同时包含字面量 \$、！和换行符的字符串。

```

1 # TODO: 实例 2 代码汇总
2 echo $'$!\n换行后的内容'

```

运行结果：单引号是规则最严格的引用方式，被单引号包裹的所有字符均保持原始字面含义；双引号属于部分解析的引用方式，被双引号包裹的内容会执行变量替换与命令替换；ANSI-C 引号是 Bash 特有的引用语法，格式为美元符号后接单引号包裹字符串，它兼具字面量安全性与转义能力。



```

MINGW64:/d/Course/System_tools_base/week1/ex/ex02
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex02 (main)
$ echo $'!\n换行后的内容'
$!\n换行后的内容
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex02 (main)
$

```

实例二运行结果

实例 3

实验题目：把上一题完成的脚本保存为文件（如 check.sh）。先运行 ./check.sh somefile，会发生什么？然后执行 chmod +x check.sh 再试一次。为什么这一步是必须的？（提示：比较 chmod 前后的 ls -l check.sh 输出）

```

1 # TODO: 实例3代码汇总
2 cat > check.sh << 'EOF'
3 #!/bin/bash
4 # 输出同时包含字面量 $、! 和换行符的字符串
5 echo $'!\n换行后的第二行内容'
6 EOF
7
8 ./check.sh somefile
9
10 ls -l check.sh
11
12 chmod +x check.sh
13
14 ls -l check.sh

```

运行结果：

预期：首次运行 './check.sh somefile'，会报错，脚本无法运行；执行 'chmod +x check.sh' 后，再次运行 './check.sh somefile'，脚本可以正常执行。

实际：直接执行 './check.sh somefile' 时，未出现预期的权限报错，脚本直接正常运行。

原因：Git Bash 会自动为脚本文件赋予可执行权限，因此即使未显式执行 'chmod +x'，脚本仍然可以运行。但在类 Unix 系统中，必须通过 'chmod +x' 显式设置可执行权限才能运行脚本。

```

MINGW64:/d/Course/System_tools_base/week1/ex/ex03
$ cat > check.sh << 'EOF'
#!/bin/bash
# 输出同时包含字面量 $、! 和换行符的字符串
echo '$!\n换行后的第二行内容'
EOF
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex03 (main)
$ ./check.sh somefile
$!
换行后的第二行内容
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex03 (main)
$ ls -l check.sh
-rwxr-xr-x 1 ZY 197121 113 Aug 25 14:52 check.sh*
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex03 (main)
$ chmod +x check.sh
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex03 (main)
$ ls -l check.sh
-rwxr-xr-x 1 ZY 197121 113 Aug 25 14:52 check.sh*
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex03 (main)
$

```

实例三运行结果

实例 4

实验题目：使用 curl 获取 [https://missing.csail.mit.edu/] 的 HTML，并通过 grep 统计列出了多少讲。（提示：找出每讲课程名称在那份 HTML 中的共性；用 curl -s 关闭进度输出。）

```

1 # TODO: 实例4代码汇总
2 curl -s https://missing.csail.mit.edu/ | sed 's/<[^>]*>//g' | grep -E
  '[0-9]+/[0-9]+/[0-9]+:' | wc -l

```

运行结果：页面中每讲课程的行都有统一的共性格式——「月 / 日 / 年份」

```

MINGW64:/d/Course/System_tools_base/week1/ex/ex04
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex03 (main)
$ cd ..
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex (main)
$ cd ex04
bash: cd: ex04: No such file or directory
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex (main)
$ mkdir ex04
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex (main)
$ cd ex04
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex04 (main)
$ curl -s https://missing.csail.mit.edu/ | grep -E '[0-9]+/[0-9]+/[0-9]+:' | wc -l
0
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex04 (main)
$ curl -s https://missing.csail.mit.edu/ | grep -c "Syllabus"
1
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex04 (main)
$ curl -s https://missing.csail.mit.edu/ | grep -E '[0-9]+/[0-9]+/[0-9]+:' | wc -l
0
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex04 (main)
$ curl -s https://missing.csail.mit.edu/ | sed 's/<[^>]*>//g' | grep -E '[0-9]+/[0-9]+/[0-9]+:' | wc -l
19
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex04 (main)
$

```

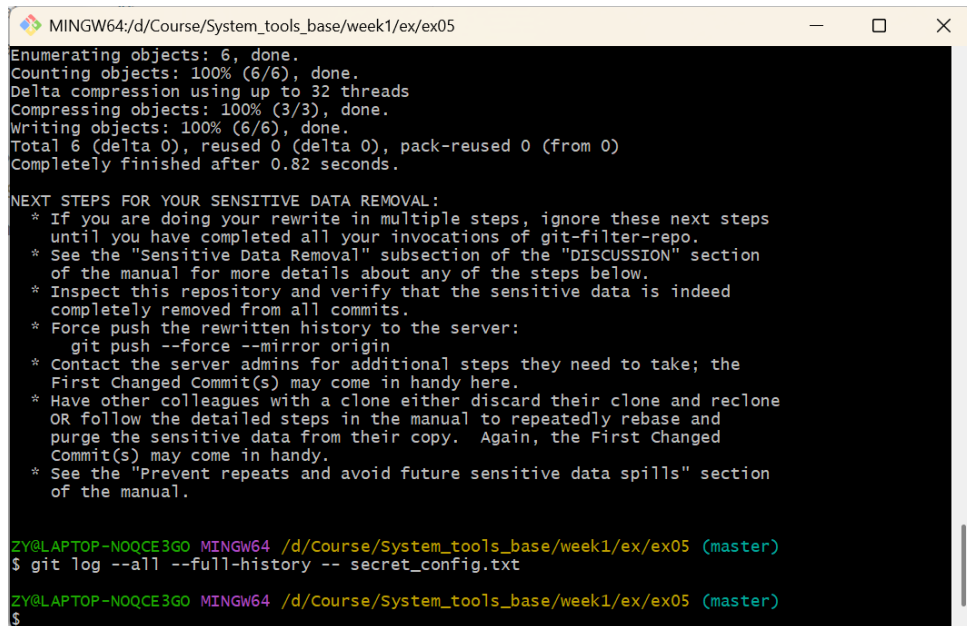
实例四运行结果

实例 5

实验题目：One common mistake when learning Git is to commit large files that should not be managed by Git or adding sensitive information. Try adding a file to a repository, making some commits and then deleting that file from history (not just the latest commit). You may want to look at [<https://help.github.com/articles/removing-sensitive-data-from-a-repository/>].

```
1 # TODO: 实例5代码汇总
2 # 1. 创建并进入测试仓库
3 mkdir git-sensitive-test
4 cd git-sensitive-test
5 git init
6 # 2. 第一次提交：项目初始化
7 echo "Git 练习项目" > README.md
8 git add README.md
9 git commit -m "init: 初始化项目，添加README"
10
11 # 3. 第二次提交：误提交敏感配置文件
12 echo "db_password=admin123" > secret_config.txt
13 echo "api_key=abcdef123456" >> secret_config.txt
14 git add secret_config.txt
15 git commit -m "feat: 添加配置文件（误包含敏感信息）"
16
17 # 4. 第三次提交：继续开发其他功能，敏感文件留在历史中
18 echo "print('hello world')" > app.py
19 git add app.py
20 git commit -m "feat: 新增业务代码文件"
21
22 # 直接从**所有提交历史**中彻底删除目标文件
23 git filter-repo --sensitive-data-removal --invert-paths --path
    secret_config.txt --force
24
25 # 验证，无任何输出
26 git log --all --full-history -- secret_config.txt
```

运行结果：



```

MINGW64:/d/Course/System_tools_base/week1/ex/ex05
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 32 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (6/6), done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Completely finished after 0.82 seconds.

NEXT STEPS FOR YOUR SENSITIVE DATA REMOVAL:
* If you are doing your rewrite in multiple steps, ignore these next steps
  until you have completed all your invocations of git-filter-repo.
* See the "Sensitive Data Removal" subsection of the "DISCUSSION" section
  of the manual for more details about any of the steps below.
* Inspect this repository and verify that the sensitive data is indeed
  completely removed from all commits.
* Force push the rewritten history to the server:
  git push --force --mirror origin
* Contact the server admins for additional steps they need to take; the
  First Changed Commit(s) may come in handy here.
* Have other colleagues with a clone either discard their clone and reclone
  OR follow the detailed steps in the manual to repeatedly rebase and
  purge the sensitive data from their copy. Again, the First Changed
  Commit(s) may come in handy.
* See the "Prevent repeats and avoid future sensitive data spills" section
  of the manual.

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex05 (master)
$ git log --all --full-history -- secret_config.txt

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex05 (master)
$

```

实例五运行结果

实例 6

实验题目：Clone some repository from GitHub, and modify one of its existing files. What happens when you do ? What do you see when running ? Run to undo what you did with . In what scenario might this be useful?git stashgit log --all --onelinegit stash popgit stash

```

1  # TODO: 实例6代码汇总
2  # 克隆公开测试仓库
3  git clone https://github.com/github/gitignore.git
4
5  # 进入仓库目录
6  cd gitignore
7
8  # 暂存当前工作区的未提交修改
9  git stash
10
11 # 查看暂存列表，确认修改已入栈
12 git stash list
13
14 # 验证工作区已恢复干净
15 git status
16
17 # 修改已有文件README.md，追加测试内容
18 echo "# test modification for git stash" >> README.md

```

```

19
20 # 查看文件修改状态
21 git status
22
23 # 查看所有引用下的提交记录（包含 stash 临时提交）
24 git log --all --oneline
25
26 # 恢复最近一次暂存的修改，并从栈中删除该记录
27 git stash pop
28
29 # 验证修改已回到工作区
30 git status
31
32 # 确认 stash 记录已移除
33 git stash list

```

运行结果：

```

MINGW64: d:/Course/System_tools_base/week1/ex/ex06/gitignore
Resolving deltas: 100% (6423/6423), done.
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex06 (main)
$ cd gitignore
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex06/gitignore (main)
$ echo "# test modification for git stash" >> README.md
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex06/gitignore (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex06/gitignore (main)
$ git stash
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it
Saved working directory and index state WIP on main: 57286c3 Merge pull request #4873 from lissy93-forks/updates-laravel-phpcache-dir
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex06/gitignore (main)
$ git log --all --oneline
3476c49 (refs/stash) WIP on main: 57286c3 Merge pull request #4873 from lissy93-forks/updates-laravel-phpcache-dir
89c72d7 index on main: 57286c3 Merge pull request #4873 from lissy93-forks/updates-laravel-php

```

实例六运行结果

实例 7

实验题目：与其他的命令行工具一样，Git 也提供了一个名为 `/.gitconfig` 配置文件 (或 dotfile)。请在 `/.gitconfig` 中创建一个别名，使您在运行 `git graph` 时，您可以得到 `git log --all --graph --decorate --oneline` 的输出结果；

```

1 # TODO: 实例7代码汇总
2 # 1. 写入全局别名，将 git graph 映射为指定的 log 命令

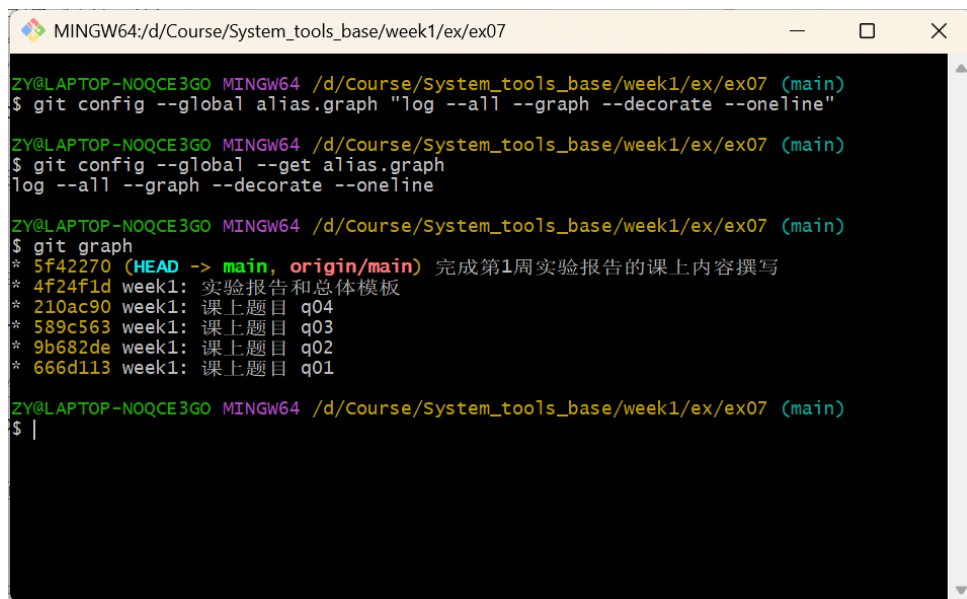
```

```

3 git config --global alias.graph "log --all --graph --decorate --
  oneline"
4
5 # 2. 验证别名已写入 ~/.gitconfig 配置文件
6 git config --global --get alias.graph
7
8 # 3. 执行别名命令，验证效果
9 git graph

```

运行结果：执行 ‘git graph’ 等价于执行完整的 ‘git log --all --graph --decorate --oneline’



```

MINGW64/d/Course/System_tools_base/week1/ex/ex07
ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week1/ex/ex07 (main)
$ git config --global alias.graph "log --all --graph --decorate --oneline"
ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week1/ex/ex07 (main)
$ git config --global --get alias.graph
log --all --graph --decorate --oneline
ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week1/ex/ex07 (main)
$ git graph
* 5f42270 (HEAD -> main, origin/main) 完成第1周实验报告的课上内容撰写
* 4f24f1d week1: 实验报告和总体模板
* 210ac90 week1: 课上题目 q04
* 589c563 week1: 课上题目 q03
* 9b682de week1: 课上题目 q02
* 666d113 week1: 课上题目 q01
ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week1/ex/ex07 (main)
$ |

```

实例七运行结果

实例 8

实验题目：设置全局忽略文件的位置，并在其中添加常见的系统/编辑器临时文件规则。

```

1 # TODO: 实例8代码汇总
2 # 1. 配置全局忽略文件的路径，写入 ~/.gitconfig
3 git config --global core.excludesfile ~/.gitignore_global
4
5 # 2. 创建全局忽略文件，并写入系统/编辑器常见临时文件规则
6 cat > ~/.gitignore_global << EOF
7 # macOS 系统临时文件
8 .DS_Store
9
10 # Windows 系统临时文件

```

```

11 Thumbs.db
12 desktop.ini
13
14 # 通用编辑器临时文件
15 *.swp
16 *.swo
17 *~
18
19 # IDE 配置目录
20 .vscode/
21 .idea/
22 EOF
23
24 # 3. 验证全局配置已生效
25 git config --global --get core.excludesfile
26
27 # 4. 测试忽略效果（在当前仓库中验证）
28 touch .DS_Store
29 git status

```

运行结果：定义的忽略规则会自动应用到所有本地仓库

```

# Windows 系统临时文件
Thumbs.db
desktop.ini

# 通用编辑器临时文件
*.swp
*.swo
*~

# IDE 配置目录
.vscode/
.idea/
EOF

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex07 (main)
$ git config --global --get core.excludesfile
C:/Users/ZY/.gitignore_global

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex07 (main)
$ touch .DS_Store

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex07 (main)
$ git status

```

实例八运行结果

实例 9

实验题目：写一行命令：仅当 /tmp/mydir 不存在时才创建它。

```

1 # TODO: 实例9代码汇总
2 # 写法1: !取反 + and 逻辑与（目录不存在时，执行创建）

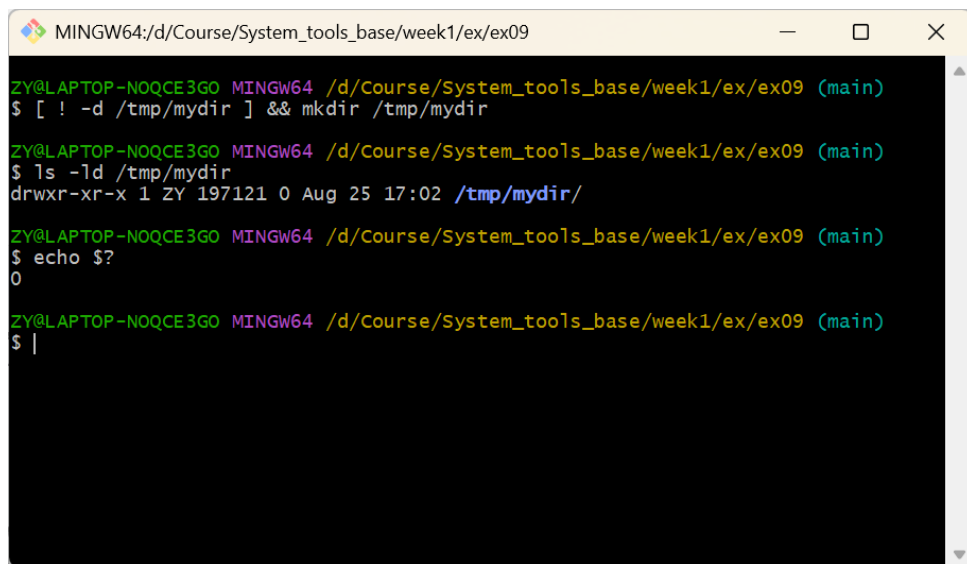
```

```

3 [ ! -d /tmp/mydir ] && mkdir /tmp/mydir
4
5 # 写法2: // 逻辑或 (目录存在判定失败时, 执行创建, 效果完全等价)
6 [ -d /tmp/mydir ] || mkdir /tmp/mydir
7
8 # 验证目录是否创建成功
9 ls -ld /tmp/mydir
10
11 # 查看上一条命令的退出状态
12 echo $?

```

运行结果：两种写法均满足「仅当目录不存在时才创建」的要求



```

MINGW64:/d/Course/System_tools_base/week1/ex/ex09
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex09 (main)
$ [ ! -d /tmp/mydir ] && mkdir /tmp/mydir

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex09 (main)
$ ls -ld /tmp/mydir
drwxr-xr-x 1 ZY 197121 0 Aug 25 17:02 /tmp/mydir/

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex09 (main)
$ echo $?
0

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex09 (main)
$ |

```

实例九运行结果

实例 10

实验题目：使用管道找出你「home 目录」中最常见的 5 种文件扩展名。（提示：组合 find、grep / sed / awk、sort、uniq -c 以及 head）。

```

1 # TODO: 实例10代码汇总
2 # 统计 home 目录下出现频次最高的 5 种文件扩展名 (屏蔽权限报错, 输出更干净)
3 find ~ -type f 2>/dev/null | awk -F. 'NF > 1 {print tolower($NF)}' |
    sort | uniq -c | sort -nr | head -5

```

运行结果：

```

MINGW64:/d/Course/System_tools_base/week1/ex/ex10
$ ls -ld /tmp/mydir
drwxr-xr-x 1 ZY 197121 0 Aug 25 17:02 /tmp/mydir/

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week1/ex/ex09 (main)
$ echo $?
0

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week1/ex/ex09 (main)
$ cd ..

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week1/ex (main)
$ cd ex10

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week1/ex/ex10 (main)
$ find ~ -type f 2>/dev/null | awk -F. 'NF > 1 {print tolower($NF)}' | sort | un
iq -c | sort -nr | head -5
64611 py
21482 js
16268 h
15309 pyc
14721 pyi

ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week1/ex/ex10 (main)
$

```

实例十运行结果

实例 11

实验题目：在脚本的 set 选项（flag）里加入 -x 会发生什么？写个简单脚本试试并观察输出。

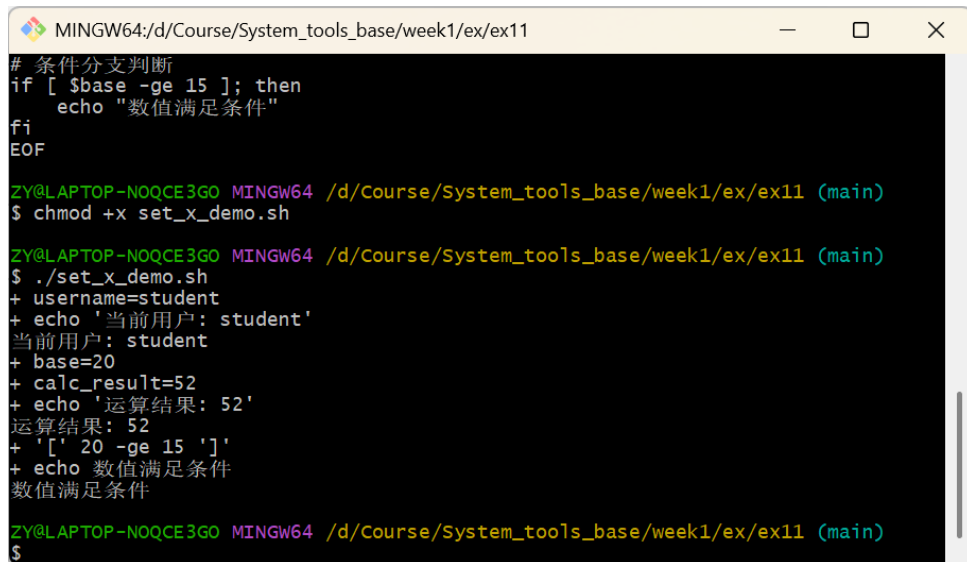
```

1 # TODO: 实例11代码汇总
2 # 1. 创建测试脚本 set_x_demo.sh
3 cat > set_x_demo.sh << 'EOF'
4 #!/bin/bash
5 set -x
6
7 # 变量定义与引用
8 username="student"
9 echo "当前用户: $username"
10
11 # 算术运算
12 base=20
13 calc_result=$((base * 3 - 8))
14 echo "运算结果: $calc_result"
15
16 # 条件分支判断
17 if [ $base -ge 15 ]; then
18     echo "数值满足条件"
19 fi
20 EOF
21

```

```
22 # 2. 赋予脚本执行权限
23 chmod +x set_x_demo.sh
24
25 # 3. 执行脚本并观察追踪输出
26 ./set_x_demo.sh
```

运行结果：



```
MINGW64:/d/Course/System_tools_base/week1/ex/ex11
# 条件分支判断
if [ $base -ge 15 ]; then
    echo "数值满足条件"
fi
EOF

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex11 (main)
$ chmod +x set_x_demo.sh

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex11 (main)
$ ./set_x_demo.sh
+ username=student
+ echo '当前用户: student'
当前用户: student
+ base=20
+ calc_result=52
+ echo '运算结果: 52'
运算结果: 52
+ '[' 20 -ge 15 ']'
+ echo 数值满足条件
数值满足条件

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week1/ex/ex11 (main)
$
```

实例十一运行结果

课后阶段提交记录

注：以上小文件夹涉及的.git 仓库已删除，为了方便大文件夹的版本控制。

[图片待补充] *week1_commit_kehou.png*

课后阶段 *commit* 记录

3 解题感悟

本周实验围绕 Shell 入门、Git 版本控制和 LaTeX 文档编辑三个主题展开，我在完成课上四道题目和课后练习的过程中收获了不少经验。

总的来说，本周实验让我对命令行工具、版本控制和文档排版都有了更扎实的动手经验，也体会到“动手做一遍”比“看一遍教程”学得更牢固。遇到问题先看报错信息、再查文档，是最有效的解决路径。